Reflection: Apple HBR RAG -- Retrieval-Augmented Generation

UTA-PGP-AIABA: Project 1 - Apple HBR RAG Module Project

Created by Brock Frary

2026-09-21 (updated 2026-10-04)

1. Project Overview

This project builds a Retrieval-Augmented Generation (RAG) prototype over an 11-page Harvard Business Review article, "How Apple Is Organized for Innovation." The scenario is a venture capital analyst who needs specific answers out of a dense report without reading the whole thing. Rather than building the RAG pipeline in isolation, the notebook answers the same three questions three different ways -- a bare LLM, a prompt-engineered LLM, and RAG -- so the value of retrieval is visible through direct, run-and-compare evidence rather than asserted. The authorship question ("who wrote this article?") turned out to be the cleanest possible demonstration of that contrast: a model with no access to the document cannot answer it, and only RAG, given the article's own text, answers it correctly with a page citation.

The notebook was submitted on 2026-09-21. A later audit against the full rubric and the course support-session deck found gaps, so the project was reopened, extended, re-run end to end on a Colab T4, and resubmitted on 2026-10-04. Section 9 records the audit and what changed; Sections 3 to 7 cover both rounds.

2. Architecture

The pipeline is a straight-through sequence, not a cyclic graph: load, clean, chunk, embed, store, retrieve, generate. The two diagrams below present the same system from two angles -- the end-to-end pipeline itself, and a side-by-side comparison of the three question-answering approaches run against it.

Figure A -- RAG Pipeline Architecture

The source PDF is loaded one Document per page (preserving page-level metadata for later citation), cleaned of repeating footers and typesetting artifacts, split into 512-token chunks with 100-token overlap, embedded, and stored in a Chroma vector index. A similarity retriever pulls the top k=8 chunks for a given question, and a chat completion generates an answer grounded in that context, returned alongside the source page numbers. Both the embedding and chat calls follow the same primary-first, automatic-fallback strategy.

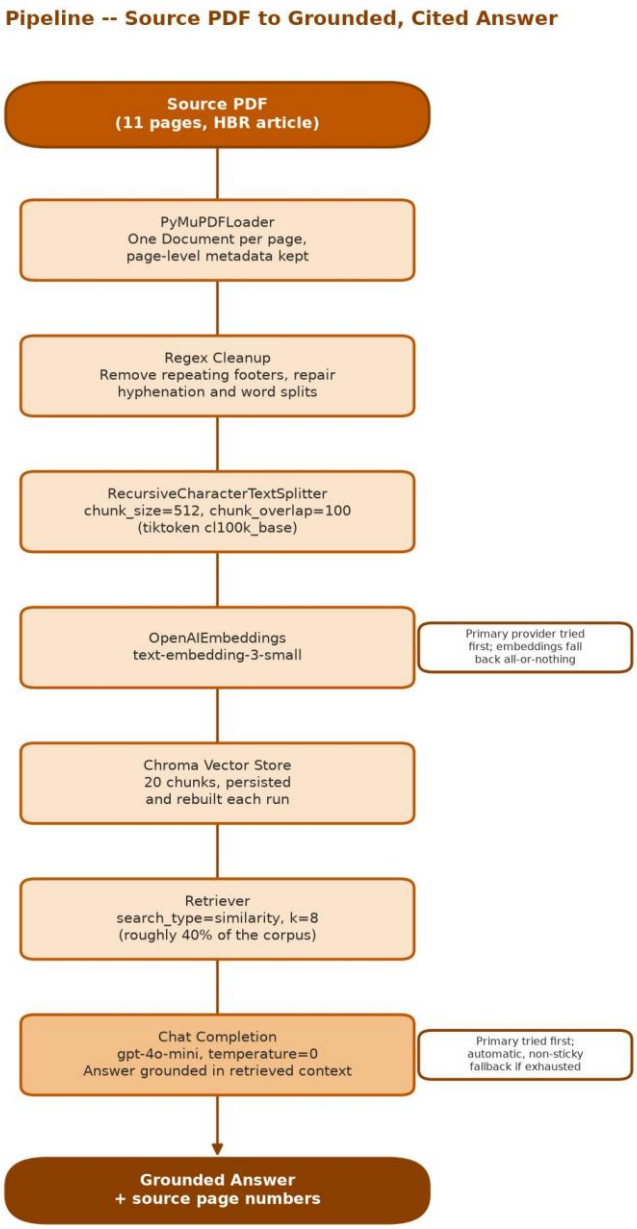
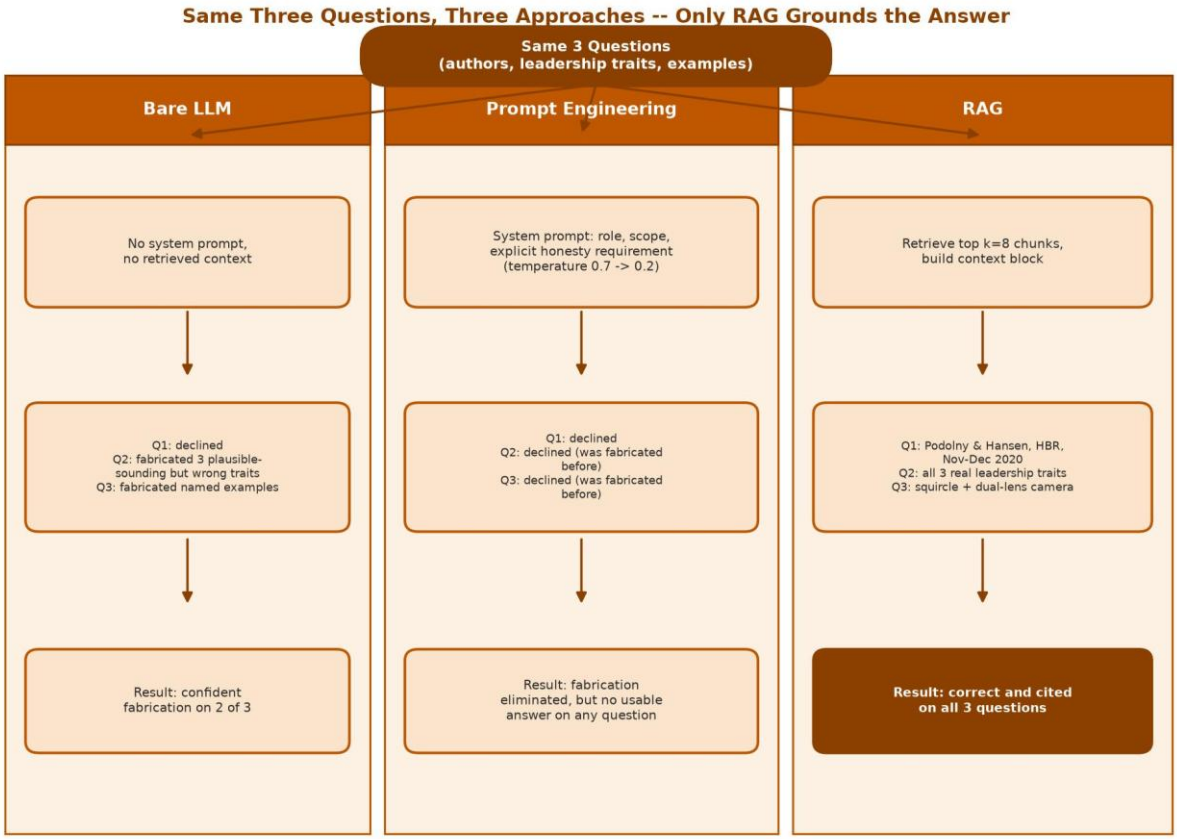


Figure B -- Three-Way Comparison

The same three questions run through all three approaches. The bare LLM either declined or fabricated plausible-sounding but wrong content; adding a system prompt with an explicit honesty requirement eliminated the fabrication but left every question unanswered; only RAG, given the article's own text, answered all three correctly with page citations. The model's raw capability never changed between runs -- what changed was whether it had the right information in front of it.



3. Design Decisions

Provider fallback strategy

Every chat and embedding call tries the primary provider first, with backoff retries on transient failures, and falls back to a secondary provider only once the primary is genuinely exhausted. The fallback is deliberately not sticky -- the next call always starts at the primary again -- so a brief primary outage does not permanently shift the notebook onto a different model. Embeddings are the one exception to per-call fallback: since vectors from two different embedding models are not comparable, a single check decides which provider embeds the entire index, so the store is never a mix of the two.

PDF cleanup approach

Inspecting the raw extracted text directly, rather than assuming a generic cleanup recipe would be sufficient, turned up five distinct artifacts: a repeating compliments-notice footer, a repeating reprint-call line, soft hyphens, zero-width spaces inside genuine compound words, and hyphens at line-wraps. The line-wrap case needed the most care: most should be joined and the hyphen dropped, but four genuine compounds (cross-functional, short-term, and two others) happen to fall on a line break and must keep their hyphen -- identified by checking every occurrence against this specific document's actual wording rather than a general rule.

Chunking parameters

chunk_size=512 tokens with chunk_overlap=100 tokens (about 20 percent). The corpus is short -- about 5,300 words after cleanup -- so a smaller chunk size gives finer retrieval granularity than one large chunk per page

would, and the overlap gives enough headroom to avoid splitting a leadership characteristic or example across a chunk boundary.

Retrieval k tuning

k=8, retrieving roughly 40 percent of the entire indexed corpus for every question. This was not the starting value -- see Section 4 for the live failure that led here. A later parameter study tested k of 3, 5, 8, and 12 across three chunk sizes. Question 3, whose evidence is spread across the article, failed at k=3 and held from k=5 upward at the chosen 512-token chunk size. k=5 would also have passed; k=8 was kept for margin at about 1,500 more context tokens per call.

RAG prompt design

The RAG prompt explicitly instructs the model to connect concrete details in the retrieved excerpts to the question even when the excerpts do not use the question's exact wording, rather than requiring a literal textual match. A narrative passage describing a specific example rarely repeats the exact words used in an abstract question about that example, so asking the model to bridge that gap in wording, not just supply the right facts, is what actually made the retrieved evidence usable -- see Section 4 for why this mattered as much as the retrieval settings themselves.

4. Challenges and Resolutions

Retrieval failure vs. synthesis failure

The RAG pipeline initially failed the question about specific innovation examples even after raising k. From the outside, a retrieval miss (the right chunk never fetched) and a synthesis failure (the right chunk fetched but the model not using it) look identical -- both come back as "the answer isn't there." Diagnosing this required inspecting the actual retrieved chunks directly, not just re-reading the final answer: the right chunk was present in context, but the model still declined to connect it to the question. The fix was combining k=8 with a revised prompt that explicitly asks the model to bridge wording differences between the question and the source text -- raising k alone was necessary but not sufficient, since the two failure modes needed two different fixes applied together.

Cross-cell CSS does not cascade in a live notebook viewer

An early attempt colored every heading level in the notebook via one shared <style> block. It appeared to work when checked with a full HTML export (nbconvert merges every cell into one continuous document), but a static export is not how the notebook is actually viewed most of the time -- in a live viewer (VS Code, JupyterLab, Colab's own view), each markdown cell renders as its own independent fragment, so the style block only ever reached the one heading sharing its own cell. The fix was scoping color to the title (H1) only, in the same cell as its own style block, and leaving every other heading at the viewer's default color -- and treating this as a reminder that a technique verified only through a merged static export is not actually verified for how the document is normally read.

Oversized title-block images in the exported HTML

The two letterhead logos were sized with a bare HTML height attribute and no width. That rendered correctly in every live notebook viewer, but broke in the actual exported static HTML: the export embeds a standard CSS bundle that resets height to auto for every image, which silently overrides a bare HTML attribute (a low-priority "presentational hint") without needing !important to do it. With no width constraint either, each logo rendered at its full natural pixel size in the file that actually gets submitted. The fix was switching to an inline style attribute on each image, which wins over a selector-based rule in an embedded stylesheet regardless of that rule's specificity --

the same category of lesson as the heading-color issue above: correct in a live viewer is not the same as verified in the actual exported artifact.

A cover image that would not render from a mounted Drive path

The report cover image was originally referenced by its absolute path inside a mounted Google Drive folder. This is not a URL the browser can fetch -- Colab's markdown renderer serves an <img src> from a real URL, not from a local or Drive-mounted filesystem path -- so the image showed a broken-image icon on the actual Colab run. This mirrors the two logos, which were external URLs from the start and always rendered correctly. The first submission accepted the gap; the resubmitted notebook removed the cover image rather than re-hosting it.

A false-positive secret-pattern match

A routine secret-leak grep for the sk- prefix (a common OpenAI key pattern) matched the substring inside the word "risk-taking," which appeared more than once across the bare LLM's hallucinated answers. Documented explicitly as a known non-issue requiring a manual look at any match, rather than trusting the pattern match alone.

An audit that checked the wrong things first

The first audit after submission compared the notebook to the written rubric and to slide screenshots, and ranked 'Output Evaluation' as a likely missing section because the label appeared on one slide. Reading the full 23-page support deck, which was already in the project folder, showed that Output Evaluation carries no points and has no description, and that the course accepts a hosted OpenAI model as the way to 'load the LLM'. Both findings were withdrawn. The real gaps were different: no RAG parameter experiments, thin prompt engineering, no embedding-model or vector-database rationale, and no next steps. Listing a folder is not reading what is in it.

Re-running the notebook broke the vector store

Running the notebook a second time in the same Colab session failed at the vector store cell with 'attempt to write a readonly database'. The cell deleted the Chroma folder while a client from the previous run still held the database file open. This was latent in the first submission, which only ever ran once. A fix was designed and tested locally (drop the old collection through the live client), but the final notebook ran without it, so it was not added afterward: the code shown in the submission should be exactly the code that ran. A plain-language 'restart the session before re-running' note covers the same case.

A gateway that intermittently rejected valid requests

The course API gateway sometimes returned a temporary 401 'no API key' error under rapid requests, even with a valid key. The existing retry logic absorbed it, which is also why narrowing the retry list to exclude 4xx errors was declined: it would have made the notebook less robust against this gateway. The retry messages appear in the output, so the notebook explains them in a note under the provider-fallback section instead of hiding them.

Answers differ between runs, so explanatory text goes stale

Model answers were not repeatable even at low temperature. Statements about specific answers, such as which examples the bare model listed or which passages RAG cited, were wrong after each fresh run until rewritten. The rule that emerged: write the explanations last, against the final run, and describe patterns that held across runs instead of exact figures where possible. A related trap: a file downloaded from Colab silently replaced a local file of the same name and discarded local edits, so downloaded runs need a different name.

5. Trade-offs

The following table summarizes the key trade-offs made during the build:

Decision	Trade-off
k=8 retrieval (roughly 40% of the corpus)	Enough breadth to surface a supporting passage even when it shares little vocabulary with the question; more context tokens sent per call than a narrower k would use
Small chunk size (512 tokens) for a short corpus	Finer retrieval granularity on an 11-page document; would need re-tuning entirely for a much longer or more varied document library
Provider fallback not sticky	A brief primary outage never permanently shifts the notebook onto a different model; every retry sequence re-tries the primary first, even if it just failed
Chroma rebuilt fresh every run	Re-executing the notebook never leaves duplicate or stale chunks behind; no benefit from a persisted index across separate runs
Notebook is Colab-only in its own narrative	Matches the actual grading and deployment environment exactly; the local execution fallback code still works but is deliberately unmentioned in the notebook's own text
Report cover image removed from the resubmitted notebook	No broken-image icon in Colab or the exported HTML; the title block no longer shows the report cover
Output Evaluation kept as an optional section	A side-by-side comparison of every approach, useful context; not a graded section, so it is labeled optional and the conclusions do not depend on it
Vector-store code fix not added to the resubmitted notebook	Displayed code equals executed code; a repeat run in the same session needs a session restart, which the notebook states
Retry list left covering 4xx errors	Rides out the gateway's transient 401 errors; a genuinely malformed request is also retried before the fallback, where it fails the same way

6. Conclusion

The project delivers a working RAG prototype, verified against live Colab runs. The three-way comparison is the real evidence for the project's central claim, not an assumption: the bare LLM declined or fabricated, prompt engineering traded fabrication for silence, and only retrieval-augmented generation produced correct, page-cited answers to all three questions. After a post-submission audit, the notebook was extended with RAG and prompt-engineering parameter experiments, embedding-model and vector-database rationale, next steps, and an optional output evaluation; it was re-run end to end on a Colab T4 with no errors and resubmitted on 2026-10-04. The audit and the changes are recorded in Section 9.

7. Personal Reflections

On testing techniques where they actually run

Two separate bugs this project (the heading-color cascade and the oversized-image sizing) shared the same root lesson: a fix that looks correct in one rendering context is not verified until it is checked in the context that actually matters. A merged HTML export and a live notebook viewer are not the same rendering model, and neither is a live

viewer the same as the exact CSS bundle an export tool embeds. The pattern that held up was treating a claimed fix as unverified until checked against the specific artifact it needed to work in, not the most convenient one to test.

On distinguishing failure modes before fixing them

The retrieval-versus-synthesis diagnosis was the single most useful debugging moment in the project. Both failure modes produce the exact same visible symptom -- an answer that says the information isn't there -- and would have been trivially easy to mis-diagnose by only reading final answers and guessing at a fix. Actually inspecting the retrieved chunks directly turned a guess into a specific, correct fix, and confirmed that raising k alone would not have been enough.

On knowing when a submission is actually final

The first submission was declared final with two cosmetic gaps knowingly accepted, and that trade-off was reasonable as far as it went. What it missed were the substantive gaps an audit against every course rubric artifact later found. Calling something done is a claim that has to be checked against the whole standard, not only the part that was easiest to see. The reopening was the right call: the cost was one more Colab run, and the result is a notebook that covers every graded section.

On reading the reference material before auditing against it

The most avoidable mistake of the resubmission was ranking a finding as critical from a tab label on one slide, when the full deck was already in the project folder. Opening and reading every reference document first would have prevented two withdrawn findings and one detour. The habit worth keeping: read the whole source, then judge.

8. Build Walkthrough -- Code

This project has no running application to screenshot the way a console-based agent project would -- everything happens inside the notebook itself. In place of screenshots, this section walks through five representative code excerpts from the notebook's own source, in pipeline order, each with a short explanation of what it does and why it is built the way it is.

Figure 1 -- Provider Fallback

Every chat call goes through the primary provider first, with backoff retries on transient failures. The fallback is used only once the primary is genuinely exhausted, and it is not sticky -- the very next call starts at the primary again.

```
RETRYABLE_ERRORS = (RateLimitError, APITimeoutError, APIConnectionError, APIStatusError)
MAX_PRIMARY_ATTEMPTS = 3
BACKOFF_SECONDS = 2

def call_chat_completion(**kwargs):
    """Call the primary provider with backoff retries, then fall back automatically if needed."""
    last_error = None
    for attempt in range(1, MAX_PRIMARY_ATTEMPTS + 1):
        try:
            kwargs.setdefault("model", PRIMARY_MODEL)
            return primary_client.chat.completions.create(**kwargs), "primary"
        except RETRYABLE_ERRORS as e:
            last_error = e
            print(f"[WARN] Primary attempt {attempt}/{MAX_PRIMARY_ATTEMPTS} failed "
                  f"({type(e).__name__}); retrying.")
            time.sleep(BACKOFF_SECONDS * attempt)
        except Exception:
            raise

    if fallback_client is None:
        raise RuntimeError(
            f"Primary provider exhausted after {MAX_PRIMARY_ATTEMPTS} attempts "
            f"({type(last_error).__name__}: {last_error}) and no fallback is configured."
        )

    print("[WARN] Primary exhausted -- switching to fallback provider for this call.")
    kwargs["model"] = FALLBACK_MODEL
    return fallback_client.chat.completions.create(**kwargs), "fallback"
```

Figure 2 -- PDF Cleanup: Repairing Typesetting Artifacts

Inspecting the raw extracted text turned up soft hyphens, zero-width spaces, and line-wrapped words. Most line-wrap hyphens should be joined and dropped, but four genuine compound words (cross-functional, short-term, and two others) happen to fall on a line break and must keep their hyphen -- identified by checking every occurrence against this specific document's actual wording, not assumed from a general rule.

```
GENUINE_HYPHEN_COMPOUNDS = {"cross-functionally", "short-term",
                            "cross-functional", "open-mindedness"}
HYPHEN_LINEBREAK_RE = re.compile(r"(\w+)-\n(\w+)")

def _dehyphenate(match):
    """Join a line-wrapped word, keeping the hyphen only for a known genuine compound."""
    before, after = match.group(1), match.group(2)
    candidate = f"{before}-{after}"
    if candidate.lower() in GENUINE_HYPHEN_COMPOUNDS:
```

```
        return candidate
    return before + after
```

Figure 3 -- Chunking Configuration

The article is short (about 5,300 words after cleanup), so a smaller chunk size than a typical corpus gives finer retrieval granularity. The 100-token overlap (about 20 percent) gives enough headroom to avoid splitting a leadership characteristic across a chunk boundary.

```
splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base",
    chunk_size=512,
    chunk_overlap=100,
)
```

Figure 4 -- Retriever Configuration

k=8 retrieves roughly 40 percent of the entire indexed corpus for every question. This started at k=4 and was raised after a live acceptance-oracle failure -- see Section 4, Challenges and Resolutions, for the diagnosis that led here.

```
retriever = vectorstore.as_retriever(search_type="similarity", search_kwargs={"k": 8})
```

Figure 5 -- Grounded RAG Response Function

Retrieves the top-k chunks, builds a context block from them, and explicitly instructs the model to connect concrete details in the context to the question even when the wording differs, rather than requiring a literal textual match. Returns the answer together with the page numbers it drew from, so each answer can be checked directly against the source.

```
def generate_rag_response(user_input, k=8, max_tokens=1024, temperature=0, top_p=0.95):
    """RAG response: retrieves the top-k most similar chunks, grounds the
    answer in them, and returns the source pages."""
    try:
        query_retriever = vectorstore.as_retriever(
            search_type="similarity", search_kwargs={"k": k})
        retrieved_docs = query_retriever.invoke(user_input)

        context = "\n\n".join(
            f"[Page {doc.metadata.get('page', -1) + 1}] {doc.page_content}"
            for doc in retrieved_docs
        )
        rag_prompt = (
            "The context below is excerpted from an article. Answer the question using only "
            "these excerpts, connecting concrete details described in them to the question "
            "even if the excerpts do not use the question's exact wording. If no answer is "
            "present anywhere in the context, say so explicitly rather than guessing.\n\n"
            f"Context:\n{context}\n\nQuestion: {user_input}"
        )

        completion, provider = call_chat_completion(
            messages=[{"role": "user", "content": rag_prompt}],
            max_tokens=max_tokens, temperature=temperature, top_p=top_p,
        )
        answer = sanitize_text(completion.choices[0].message.content)
        source_pages = sorted({doc.metadata.get("page", -1) + 1 for doc in retrieved_docs})
        print(f"[INFO] served by: {provider}  |  source pages: {source_pages}")
```

```
        return answer, source_pages
    except Exception as e:
        return f"[ERROR] RAG response failed: {type(e).__name__}: {e}", []
```

9. Rubric Audit and Remediation Plan

On 2026-10-04 the submitted notebook was audited against the written rubric, the project description, and the Great Learning Project Evaluation Rubric slides. The submission is accurate and fully executed -- every fact in the RAG answers was confirmed in the source PDF -- but it leaves several rubric items only partly covered. The project was reopened to fix them and resubmit. Status is tracked in ROADMAP.md, Stage 8.

Definition of done

Every finding is fixed or explicitly declined with a reason; the notebook is re-run top to bottom in Colab on a T4 with no errors; a new single .html is exported; and the same audit is repeated with no partly-covered rubric rows and no finding above low criticality. The final pipeline configuration (512/100 chunks, k=8, model, provider fallback) is out of scope.

Audit result by rubric criterion

Criterion (points)	Result	Gap
Q&A, generic prompts (10)	Met	Cell 27 misstates the bare-LLM Q3 output.
Prompt engineering (14)	Partial	One prompt, one parameter change; prompt-design factors and creativity-vs-specificity parameters not covered.
Data preparation for RAG (12)	Partial	Embedding-model choice and vector-database rationale not stated.
Q&A using RAG (12)	Partial	One configuration only; no chunking, retriever, or LLM parameter experiments. Cell 56 says two Q3 examples; the output shows four.
Output Evaluation	Not graded	Appeared only as a tab label; the full deck shows no points and no description.
Actionable insights (4)	Partial	No explicit next steps.
Presentation / quality (8)	Mostly met	Small text inconsistencies; missing sections above.

Remediation plan, in order of criticality

Step	Work	Approval
1	RAG experiments: 3 chunk configurations x 3 values of k at temperature 0, then temperature 0 vs 0.7 on the best combination, scored against the expected facts, with observations.	New cells only
2	Prompt engineering: two more prompt variants (format-explicit, few-shot), a temperature / top_p / max_tokens comparison, and a prompt-design factors section.	New cells only
3	Output Evaluation: scored table across the approaches, added as an optional extra once the deck	New cells only

	showed it is not a graded section.	
4	Embedding-model rationale, vector-database rationale, and a next-steps section.	New text
5	Correct the Q3 example count, the bare-LLM Q3 description, the artifact count in the cleanup section, and process wording.	Exact wording first
6	Clarify how the LLM is 'loaded' (API client vs local model).	Exact wording first
7	Retry-list error classes; page-attribution granularity (recommend declining and documenting).	Approval required
8	Colab T4 re-run, new .html export, repeat the audit, update the roadmap, log, README, and this document.	None

Outcome

All eight steps were completed on 2026-10-04 and the audit was repeated against the six graded sections. The notebook was re-run top to bottom on a Colab T4 with no errors, and the explanatory text was rewritten against that final run's outputs. Two audit findings were withdrawn after the full support-session deck was read: Output Evaluation is not a graded section (it was kept as an optional extra), and loading a hosted OpenAI model satisfies the 'load the LLM' requirement.

Criterion (points)	**Result after remediation**
Q&A, generic prompts (10)	Met
Prompt engineering (14)	Met: rule-based and few-shot prompts, parameter study
Data preparation for RAG (12)	Met: embedding and vector-database rationale added
Q&A using RAG (12)	Met: 12-setting chunking and k grid, generation-parameter test
Actionable insights (4)	Met: Next Steps added
Presentation / quality (8)	Met: text errors corrected, sections complete

Lesson

The first pass checked the work against the written rubric but not the slide-level guiding questions, which hold requirements the written rubric only implies, and it missed that the full deck was already in the project folder. A submission should be checked against every rubric artifact the course provides, after actually opening each one.

Two practical lessons came from the re-run. Explanatory text that cites specific model output must be written against the final run, because answers differ between runs even at low temperature. And a file downloaded from Colab silently replaces a local file of the same name, so downloaded runs need a different name.